

ScriptLens — Status Report

Prepared for: Product planning and writer outreach

Date: July 2026

Version: Engine + internal tools (no public web app yet)

1. Executive summary

ScriptLens is a **screenplay continuity and scene-dependency engine**. It reads a script, builds a map of how scenes connect (characters, props, locations), finds story logic problems, and can **simulate removing one scene** to show what would break downstream.

What exists today: A working Python engine, command-line tools, automated tests, a 40-script error corpus with **100% detection** on planted mistakes, and a PDF-to-Fountain cleanup pipeline proven on Hollywood scripts (Carrie, American Pie).

What does not exist yet: A web upload interface, Chrome extension, user accounts, billing, or a polished product writers can sign up for. Those are specified in design documents but not built.

Bottom line: The **brain works**. The **product wrapper does not**. Before showing this to paying writers, Hollywood PDFs need cleanup, and a simple upload-and-report website needs to be built.

2. What ScriptLens is trying to be

ScriptLens is **not** a general AI writing assistant. It does not rewrite dialogue or generate scenes.

It is a **hybrid analysis tool**:

- **Mostly deterministic rules** — pattern matching and logic for ages, props, counts, relationships, and scene links.
- **Light semantic checks** — spaCy similarity on a small set of harder cases.
- **Small LLM only where needed later** — reserved for ambiguous candidates, not whole-script analysis.

Target user: Screenwriters who want to catch **continuity slips** (wrong age, prop changes hands impossibly, timeline breaks) and understand **what happens if they cut a scene** — without editing their original file.

Business model (planned): Thin client (browser or extension) talking to a server-side engine. Writers on low-RAM laptops should not run heavy NLP locally.

3. What the application CAN do today

3.1 Upload and analyse a script (command line)

You can point the tool at a screenplay file and get a structured report:

Fountain / plain text	Full support — best quality
PDF	Supported with automatic conversion to cleaned Fountain text before analysis
Final Draft (.fdx)	Not directly supported yet — export to Fountain or PDF first

Commands (developer / internal use):

```
venv\Scripts\python.exe run_scriptlens.py your_script.fountain
venv\Scripts\python.exe run_scriptlens.py your_script.pdf --save-fountain output.fountain
venv\Scripts\python.exe scripts\convert_pdf_to_fountain.py your_script.pdf
```

The report includes:

- Scene count, character list, prop list
- Scene dependency graph (how scenes link through shared characters, props, locations)
- **Orphan scenes** — scenes nothing later depends on (possible cuts or weak links)
- **High-risk scenes** — scenes many later scenes depend on (dangerous to cut)
- **Contradiction list** — possible story logic errors with scene numbers and explanations
- **Health score** — a single 0–100 summary (penalises contradictions and orphans)

3.2 Story continuity detection

The engine checks for many types of mistakes. On our **planted error corpus** (100 intentional errors across 40 short scripts), **all 100 are detected** with only **2 false alarms** across the whole corpus.

Object ownership	A prop is held by one character, then another uses it with no explanation	100% caught
Object identity	Same prop, different colour or material (leather bag → canvas bag)	100% caught
Object referent swap	Same action thread, different noun (blanket → towel in one scene)	100% caught (new)
Character age	Character age stated differently in two scenes	100% caught
Name consistency	Same character spelled two ways (Chris vs Chius)	100% caught
Numeric count	“Three hostages” then “all four”	100% caught
Date / year	Timeline year contradictions	100% caught
Character knowledge	Character knows something before they could learn it	100% caught
Relationship facts	Daughter vs niece, etc.	100% caught
Location continuity	Place described inconsistently	100% caught
Medical state	Injury side or recovery contradictions	100% caught
Character traits	Job/role contradictions (novelist vs poet)	100% caught
Character alive/dead	Dead in one scene, alive later	Supported
World rules	Story rules violated	Supported
Fact consistency	General fact clashes	Supported

Important: Corpus performance is on **short Fountain scripts with deliberate errors**. Real Hollywood PDFs behave differently (see Section 5).

3.3 Scene dependency graph and “simulate cut”

The engine builds a **directed graph** of scenes:

- **Continuity edges** — same character, prop, or location reappears
- **Fact edges** — later scenes rely on established facts
- **Causal edges** — dialogue references prior events

From this graph you can ask: “**If I remove scene 12, what breaks?**”

This logic exists in the engine today (`get_delete_impact`). It is **not yet exposed** in a web UI with one-click “Simulate cut,” but the UX spec and mockups are complete.

3.4 PDF ingestion and noise reduction

PDF screenplays are messy. Camera directions (`ANGLE ON JIM`), page numbers, and broken line breaks often get misread as character names.

Automated pipeline (working):

1. **Extract** PDF to plain text
2. **Clean** — merge action lines, demote camera slugs
3. **Refine (optional)** — script-specific rules for benchmark scripts
4. **Analyse** with a stricter `pdf_benchmark` profile that avoids numeric false positives

Proven on Hollywood pilots:

Carrie (1976)	267	29	0 (after cleanup)
American Pie (1999)	74	19	0 (after cleanup)

PDF uploads now **automatically run the clean conversion** before analysis.

3.5 Automated quality gates (CI)

- **40-script planted corpus** — must stay at 100% recall, ≤ 4 false positives
- **21+ unit test files** covering age, props, relationships, PDF loading, referent swap, etc.
- **Hollywood benchmark folder** — manual false-positive review in progress

3.6 Design and product specification (written, not coded)

Completed artifacts:

- **UX spec v1** — upload, 3-panel workspace, simulate cut, go-to-scene on contradictions
- **UI mockups** — main view, simulate active, contradictions panel
- **Architecture notes** — thin client, FastAPI backend, Redis cache (planned)
- **Writer outreach materials** — cheat sheets, error guides, cleanup guide

4. What the application CANNOT do today

4.1 No public product yet

Web upload page	Not built
-----------------	-----------

FastAPI / REST API	Not built
User accounts / login	Not built
Chrome extension	Not built (v1.1 in spec)
Payment / billing	Not built
Cloud hosting / deployment	Not built

Writers **cannot** sign up and use ScriptLens in a browser today. Everything runs on your machine via Python.

4.2 Dialogue and formatting (explicitly v2)

The engine **does not** reliably catch:

- Wrong dialogue speaker (character A's line under character B's name)
- Fountain formatting errors
- Parenthetical mistakes (`to Robert`) when Robert is not in the scene
- Stage direction vs dialogue attribution ping-pong

These are logged in the backlog for **version 2**, not the current focus.

4.3 Hollywood PDFs without cleanup

Raw PDFs from produced scripts often produce **bad cast and prop lists** even when contradiction count looks low.

Example from raw Hollywood batch (6 scripts, no cleanup):

Citizen Kane	20	Life-stage ages flagged (often false alarms on biographical scripts)
Almost Famous	11	Age and trait noise
Batman Begins	9	Name drift + object noise
American Beauty	8	POV parsed as a prop
Conclave	4	Age false positives
American Pie	0 flags but 74 junk characters	Misleadingly "clean"

Do not demo raw PDF output to writers. Cleanup or Fountain-first upload is required.

4.4 Prop tracking at full-script scale

Even after cleanup, prop lists can stay noisy (~160 entries on American Pie). Many are dialogue fragments or sound cues, not real story props.

The engine **does not** track every object in a 200-scene script equally well. It is strong on:

- Props introduced in ALL CAPS in action
- Props tied to handling verbs (grabs, holds, sewn into, etc.)

- **Salient** recurring props

It is weak on:

- Background objects mentioned once in lowercase
- Synonym swaps **across scenes** (blanket in scene 64, towel in scene 65) — only **same-scene** referent swap is implemented
- Every item in a crowded production script without a salience filter

4.5 Writer-material edge cases (partially open)

From review of real writer scripts (12 Years a Slave material):

Slugline says NIGHT, action says “Later in the Morning”	Not detected yet (E5 backlog)
Wrong character name + pronoun mismatch in action (Eliza / “His cries”)	Not detected yet (E7 backlog)
Window opens before character opens it (in-scene order)	Not detected yet (E2 backlog)
Blanket → towel same scene	Now detected (E6 shipped)

4.6 Not an LLM product

ScriptLens does **not** offer:

- “Fix my script” rewrites
- General story notes or coverage
- Character voice analysis
- Market / comps / pitching advice

4.7 Scanned PDFs

Image-only PDFs (no selectable text) require OCR first. The loader will fail with a clear error.

4.8 Final Draft native integration

No plugin for WriterDuet, Arc, or Final Draft yet. B2B integration is a **post-D2C** path.

5. Measured performance — honest numbers

5.1 Planted corpus (controlled test)

- **40 scripts, 100 planted errors**
- **Recall: 100%** (100/100 found)
- **False positives: 2** across entire corpus
- **Precision: 98%**

This proves the **detector logic works** on known error types in clean Fountain input.

5.2 Hollywood clean benchmark (real PDFs)

- **7 PDFs** collected (Carrie, American Pie, Batman, Kane, Conclave, American Beauty, Almost Famous)
- **2 fully cleaned** (Carrie, American Pie) — demo-ready cast lists
- **5 awaiting cleanup** — raw analysis not writer-trustworthy

- **52 total flags** on 6 raw PDFs — mostly expected false positives from PDF noise and biographical age jumps

5.3 Product readiness score (estimated)

Engine technically works	~85%	Core done, tested
Writer-trustworthy on real scripts	~65%	Needs cleanup + Fountain-first
Demo-ready product (web UI)	~15%	Spec only
Revenue-ready business	~30–35%	Distribution not started

6. Architecture today vs planned

Today

```
[You] → run_scriptlens.py / Python API → scriptlens_analyser.py
                                     ↓
                             scene_dependency.py + plot_contradiction.py
                                     ↓
                             Text report + JSON
```

All processing is **local**. One shared spaCy English model (~small, no word vectors).

Planned (UX spec v1)

```
[Writer browser] → upload PDF/Fountain → FastAPI server
                                     ↓
                             convert PDF → analyse → cache in Redis
                                     ↓
                             JSON + rendered workspace (3 panels)
```

Original file **never modified**. Simulate cut is a **preview** only.

7. Detailed analysis — strengths and risks

Strengths

1. **Real engine, not a wrapper** — Thousands of lines of domain-specific logic, not “send script to GPT.”
2. **Test discipline** — 100 planted errors with CI gate; regressions are caught.
3. **Differentiated feature** — Simulate cut + dependency graph is rare in writer tools.
4. **PDF path exists** — Automatic conversion reduces the worst PDF noise.
5. **Clear v1 scope** — Continuity + simulate cut; dialogue format deferred to v2.

Risks

1. **PDF trust gap** — Raw PDF demos will embarrass you in front of writers.
2. **Manual cleanup does not scale** — Carrie and American Pie needed script-specific whitelists; need more automation or Fountain-first messaging.

3. **No product shell** — Cannot charge money without upload UI and hosting.
 4. **False positive sensitivity** — Writers forgive missed errors more than wrong accusations; Hollywood biographical scripts trigger age flags.
 5. **Solo distribution** — Building is ~60% of the journey; writer interviews and launch are untested.
-

8. Next course of action (plain English)

Phase A — Finish input quality (1–2 weeks)

Goal: Never show a writer a broken cast list.

1. Run PDF → clean → refine pipeline on **Batman, Citizen Kane, Conclave, American Beauty, Almost Famous**.
2. Complete false-positive review in `CLEAN_FP_LOG.md` for each.
3. Default messaging: “**Upload Fountain for best results; PDF supported with automatic cleanup.**”
4. Hide or de-emphasise the props panel until prop noise drops.

Phase B — Ugly but real web demo (2–3 weeks)

Goal: A URL you can send to 10 writers.

1. Build **FastAPI** with three endpoints: upload, analyse, simulate-cut.
2. Single-page web UI: upload → scene list → contradiction list → click scene.
3. Server-side only; no extension yet.
4. Use **cleaned Fountain or auto-cleaned PDF** only in demos.

Phase C — Writer validation (2 weeks, parallel with B)

Goal: Learn if anyone will pay.

1. Talk to **10 screenwriters** (not engineers).
2. Show simulate cut + 2–3 real contradiction catches on **their** script or a cleaned Hollywood sample.
3. Ask: “Would you pay \$15/month for this?” Track yes/no/maybe.
4. Do **not** fine-tune the engine based on one script — wait for 3+ similar examples.

Phase D — Corpus and engine backlog (ongoing, gated)

Goal: Close writer-material gaps without breaking CI.

1. **E5** — Slugline time vs action time (NIGHT vs “Later in the Morning”).
2. **E7** — Wrong character / pronoun in action block.
3. **E2** — In-scene ordering (window opens before “opens the window”).
4. Only implement when **3+ planted examples** exist.

Phase E — Soft launch (after A + B + 3 positive writer signals)

1. Privacy policy + basic billing (Stripe).
2. Landing page with demo video (2 minutes: upload → issue → simulate cut).
3. Soft launch to r/screenwriting, Stage 32, private writer groups.
4. Target: **\$300–700 MRR in 3 months** (ambitious but plausible with hustle).

What NOT to do next

- Do not build the Chrome extension before the web demo works.
- Do not tune regex for one writer script without corpus backup.
- Do not show raw Hollywood PDF analysis to writers.
- Do not pursue B2B (WriterDuet/Arc API) before D2C proof.

9. Summary table

Analyse Fountain script	Yes (CLI)	Web upload
Analyse PDF	Yes with auto-cleanup	Same, with quality warning
Continuity contradictions	Yes, 14+ types	Same + go-to-scene UI
Simulate cut	Engine yes, UI no	One-click preview
Scene dependency graph	Yes	Visualise in sidebar
Orphan / high-risk scenes	Yes	List in UI
Dialogue speaker errors	No	v2
Writer signup / billing	No	Stripe + accounts
Hollywood PDF out-of-box	No	Cleanup + benchmark
100% planted corpus CI	Yes	Keep green

10. Document history

Report type	Internal status / investor-writer briefing
Engine baseline	100/100 planted errors, 2 FPs
Hollywood pilots	Carrie + American Pie cleaned
Product UI	Spec + mockups only
Recommended focus	Cleanup remaining PDFs → FastAPI demo → 10 writer interviews

ScriptLens — “Upload your script, see what breaks, simulate the cut — without changing your file.”